

AI-Powered Banking Transaction Processing Pipeline

HW6 Final Capstone · Roman Popyk · July 2026

Project Goal

Build a working transaction processing pipeline powered by AI workflow agents.

The system validates raw banking transactions, scores valid transactions for fraud risk, and generates a final processing summary.

Main deliverables: pipeline code, tests, coverage gate, MCP integration, front-end dashboard, and documentation.

High-Level Architecture

```
sample-transactions.json
|
v
Orchestrator
|
v
shared/input
|
v
Transaction Validator
|
v
shared/processing
|
v
Fraud Detector
|
v
shared/output
|
v
Reporting Agent ---> shared/results/pipeline-summary.json
```

File-Based Pipeline Protocol

Each stage communicates through JSON files instead of direct function calls.

Why this design works:

- Stages are decoupled and easy to replace
- Each stage has clear input and output folders
- Failed transactions can be written immediately to results
- The protocol is transparent and easy to debug

```
shared/input -> shared/processing -> shared/output -> shared/results
```

Stage 1: Transaction Validator

The validator protects the rest of the pipeline from bad data.

Checks:

- Required fields are present
- Amount is positive and parsed with `decimal.js`
- Currency exists in the ISO 4217 code set
- Timestamp is valid ISO 8601

Output: valid transactions move forward; invalid transactions are rejected with a reason.

Stage 2: Fraud Detector

The fraud detector assigns a numeric risk score to each validated transaction.

RISK FACTOR	SCORE
Amount greater than 10,000	+40
Cross-border transaction	+30
Off-hours transaction	+20
Wire transfer	+10

Transactions with score **60 or higher** are marked `FLAGGED_FOR_REVIEW`.

Stage 3: Reporting Agent

The reporting agent creates the final pipeline summary.

Reads:

- Approved and flagged transactions from `shared/output`
- Rejected transactions from `shared/results`

Writes:

- Total transaction count
- Approved, flagged, and rejected counts
- Per-currency counts and amount totals

Workflow Agents Used

AGENT	RESPONSIBILITY	DELIVERABLE
Spec	Define behavior before coding	<code>specification.md</code>
Code	Build the pipeline	TypeScript stages + dashboard
Tests	Verify behavior	Jest tests + coverage hook
Docs	Explain the system	README, HOWTORUN, presentation

Demo Results

Running `npm run pipeline` processes 8 sample transactions.

8

Total transactions

6

Approved transactions

0

Flagged for review

2

Rejected transactions

Rejection Examples

Two sample records are rejected by the validator.

TRANSACTION	REASON
TXN006	Unknown ISO 4217 currency code: XYZ
TXN007	Amount must be positive: -100.00

Rejected records are written to `shared/results` with a machine-readable `reason` field.

Front-End Dashboard

The project includes a simple web dashboard running at `http://localhost:3000`.

Dashboard features:

- Run the pipeline from the browser
- View total, approved, flagged, and rejected counts
- Inspect per-currency totals
- Review transaction status, risk score, and rejection reason

Run it with:

```
npm run frontend
```

MCP Integration

Two MCP servers are configured in `mcp.json`.

SERVER	PURPOSE
<code>context7</code>	Look up current docs for libraries and framework patterns
<code>pipeline-status</code>	Query live pipeline results from an MCP client

The custom server is implemented in `mcp/server.ts`.

Custom MCP Server

The `pipeline-status` server exposes two tools and one resource.

TYPE	NAME
Tool	<code>get_transaction_status(transaction_id)</code>
Tool	<code>list_pipeline_results()</code>
Resource	<code>pipeline://summary</code>

This lets AI tools query pipeline state without manually opening result files.

Quality Gate

A Husky `pre-push` hook runs the coverage check before every push.

```
git push
  -> .husky/pre-push
  -> npm run test:coverage
  -> block push if coverage is below 80%
```

Coverage thresholds are set in `jest.config.js` for branches, functions, lines, and statements.

Test Coverage

The test suite contains **68 tests** across validators, runners, utilities, logging, fraud logic, and reporting.

100%

Statements, branches, functions, and lines

7

Unit test files

Tests isolate file I/O with temporary or mocked directories where needed.

Lessons Learned

- Monetary values must use decimal arithmetic, not native floating point
- File-based protocols make multi-stage pipelines easier to inspect and debug
- Automated hooks prevent quality regressions before code reaches the remote branch
- MCP servers are a practical way to expose project state to AI coding tools
- AI agents speed up implementation, but tests and manual review still matter

Thank You

Roman Popyk · HW6 Final Capstone

```
npm install  
npm run pipeline  
npm run frontend  
npm run test:coverage  
npm run mcp
```